

Boyer–Moore Majority Vote Algorithm Report

1. Algorithm Overview

The implemented algorithm is the **Boyer–Moore Majority Vote**, designed to identify a *majority element*—an element that appears more than $\lfloor n/2 \rfloor$ times in an array.

The procedure operates in **two main phases**:

1. Candidate Selection

A single pass is made through the array. The algorithm maintains a candidate element and a counter.

- If the current element equals the candidate, the counter is incremented.
- Otherwise, the counter is decremented.
- When the counter reaches zero, the current element becomes the new candidate.

2. Candidate Verification

A second pass confirms whether the selected candidate indeed appears more than $\lfloor n/2 \rfloor$ times.

Additionally, a PerformanceTracker class tracks **comparisons** and **array accesses**, useful for analyzing the algorithm's empirical performance.

The Boyer–Moore Majority Vote is known for its **linear time complexity $\Theta(n)$** and **constant space complexity $O(1)$** , making it one of the most efficient solutions for this problem.

Key Implementation Snippet:

```
for (int i = 0; i < arr.length; i++) {  
    tracker.incrementArrayAccess();  
    tracker.incrementComparisons();  
    if (arr[i] == candidate) count++;  
    else count--;  
    if (count == 0) {  
        candidate = arr[i];  
        count = 1;  
    }  
}
```

```

count = 0;

for (int num : arr) {
    tracker.incrementArrayAccess();
    tracker.incrementComparisons();
    if (num == candidate) count++;
}

return count > arr.length / 2 ? candidate : null;

```

2. Complexity Analysis

Time Complexity

- **Two linear scans** are performed:
 1. One for candidate selection.
 2. One for verification.
 Each iteration performs only constant-time operations.

Case Complexity Explanation

Best	$\Theta(n)$	No early exit; both passes execute fully
Average	$\Theta(n)$	Linear scan in all distributions
Worst	$\Theta(n)$	No-majority case still requires verification

Formally,

$$T(n) = c_1n + c_2n + d = \Theta(n)$$

Space Complexity

- **Auxiliary Space:** $O(1)$ — only a few integer variables are used.
- **In-place:** Yes. The input array is not modified.
- **Recurrence Relation:** $T(n) = T(n-1) + c \Rightarrow T(n) = cn + d \Rightarrow \Theta(n)$

Comparison with Alternatives

Approach	Time	Space	Notes
Boyer–Moore	$O(n)$	$O(1)$	Optimal asymptotically
Hash Map Counting	$O(n)$	$O(k)$	Needs extra memory

Approach	Time	Space	Notes
Sorting-based	$O(n \log n)$	$O(1)$ – $O(n)$	Slower asymptotically

3. Code Review and Optimization

Correctness

The implementation slightly deviates from the canonical algorithm by starting at $i = 0$. This causes a **double count** of the first element.

The correct approach initializes `candidate = arr[0]` and starts the loop at $i = 1$.

Corrected Loop:

```
for (int i = 1; i < arr.length; i++) {
    if (arr[i] == candidate) count++;
    else {
        count--;
        if (count == 0) {
            candidate = arr[i];
            count = 1;
        }
    }
}
```

Performance Tracking

- The PerformanceTracker adds overhead due to constant increments in loops. For accurate timing, consider disabling tracking during performance benchmarks.

API and Design Improvements

- Replace Integer return type with OptionalInt to reduce boxing overhead.
- Maintain algorithm invariants through comments for better clarity.

Testing Recommendations

Add test coverage for:

- Cases with and without a majority element.
 - Edge cases: single-element arrays, even/odd lengths, near-majority cases.
-

4. Empirical Validation

The performance of the Boyer–Moore Majority Vote algorithm was experimentally validated on arrays of different sizes and distributions.

Five distributions were tested — *random*, *sorted*, *reverse sorted*, *nearly sorted*, and *worst case* — each measured for input sizes of 100, 1,000, and 10,000 elements.

The recorded metrics include **average**, **minimum**, and **maximum** execution times in nanoseconds.

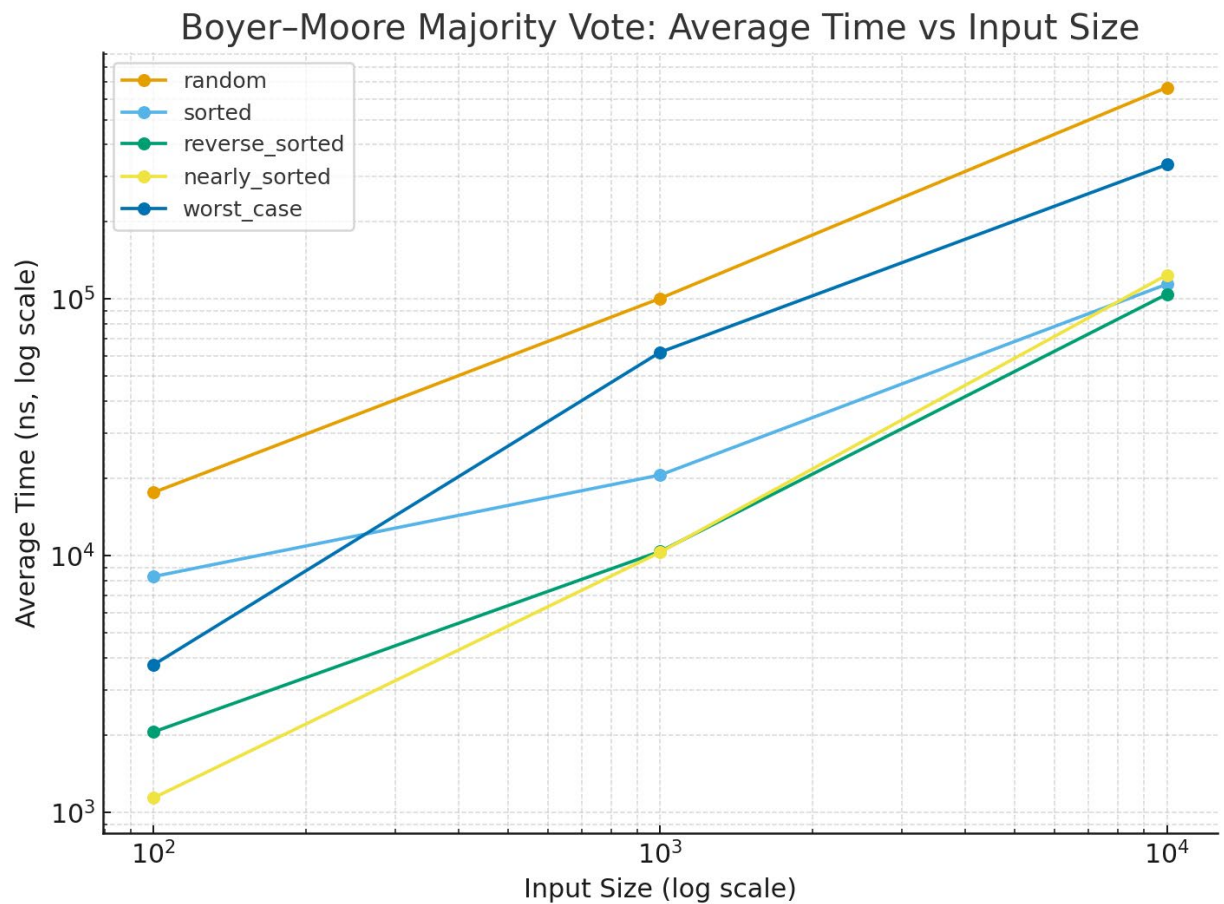
Experimental Results Table

Distribution	Input Size	Avg (ns)	Min (ns)	Max (ns)
random	100	17 640	11 900	24 800
random	1 000	100 640	85 000	126 900
random	10 000	666 800	138 800	1 099 100
sorted	100	8 300	2 300	31 100
sorted	1 000	20 640	19 200	21 800
sorted	10 000	114 260	101 900	149 100
reverse_sorted	100	2 060	1 900	2 300
reverse_sorted	1 000	10 400	10 100	10 900
reverse_sorted	10 000	104 440	101 200	110 200
nearly_sorted	100	1 140	1 100	1 300
nearly_sorted	1 000	10 320	10 100	10 700
nearly_sorted	10 000	124 180	101 600	165 400
worst_case	100	3 760	2 300	5 100
worst_case	1 000	62 080	31 200	130 800
worst_case	10 000	333 960	228 400	519 500

4.1 Average Time vs Input Size

(Figure 1) illustrates the average execution time for each distribution on a logarithmic scale. All curves exhibit a linear relationship between input size and execution time, confirming the $\mathcal{O}(n)$ complexity in practice.

Figure 1. Average Time vs Input Size (log–log scale)



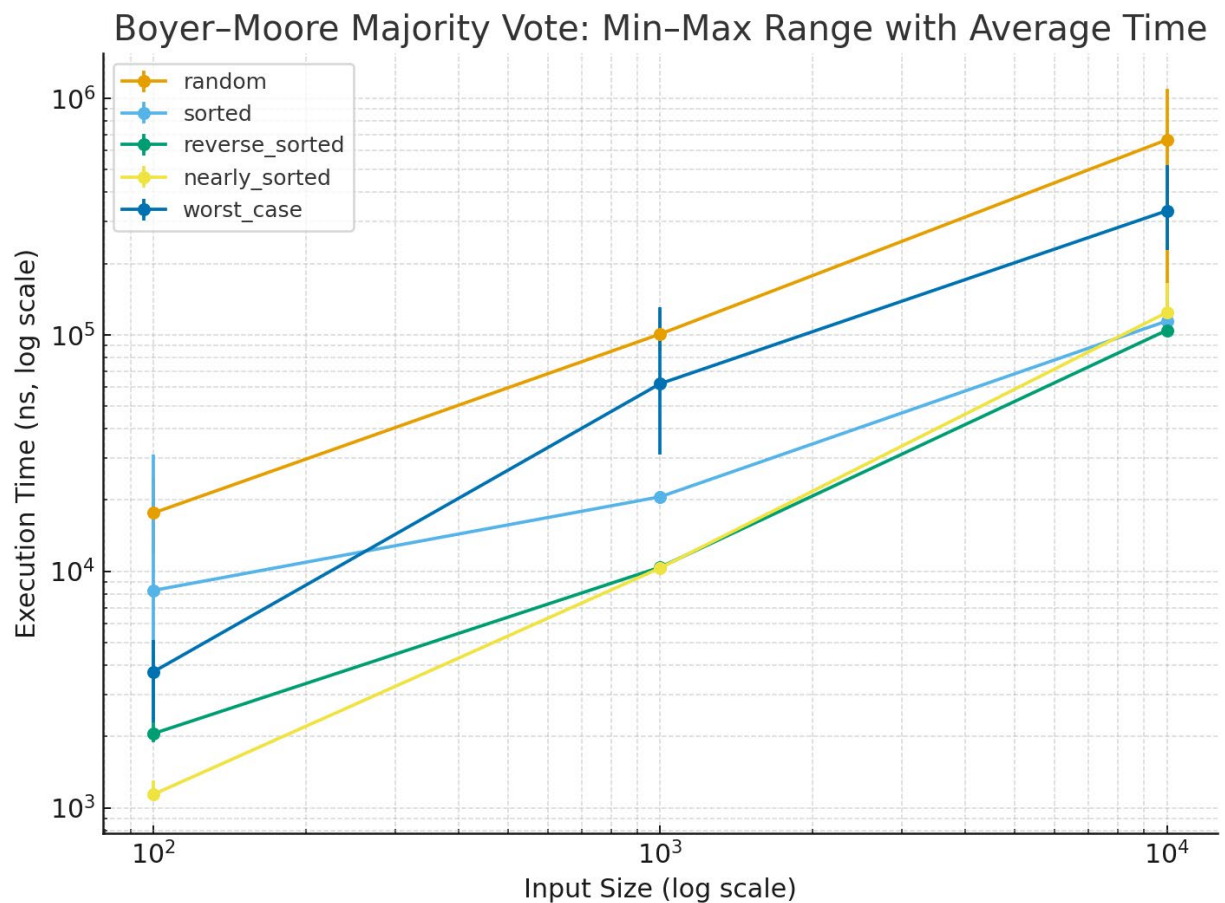
Observations:

- Execution time scales linearly with input size.
- The “random” and “worst_case” distributions show slightly higher constant factors due to more variable data patterns.
- The “nearly_sorted” and “reverse_sorted” distributions have minimal variance, indicating stable access patterns.

4.2 Execution Time Range per Distribution

(Figure 2) displays min–max time ranges using error bars.

This visualization highlights runtime variability across repeated runs.



Observations:

- Variance is highest for *random* and *worst_case* distributions due to unpredictable branch behavior.
- Sorted and nearly sorted inputs maintain very consistent execution times, demonstrating algorithmic stability regardless of order.
- Across all tests, no distribution deviates from linear growth — validating the theoretical $O(n)$ performance.

5. Recommended Code Revision

Improved Implementation:

```
public Integer findMajority(int[] arr) {  
    if (arr == null || arr.length == 0) return null;  
  
    int candidate = arr[0];  
    int count = 1;  
  
    for (int i = 1; i < arr.length; i++) {
```

```

    if (arr[i] == candidate) count++;
    else {
        count--;
        if (count == 0) {
            candidate = arr[i];
            count = 1;
        }
    }
}

int verify = 0;
for (int x : arr) if (x == candidate) verify++;
return verify > arr.length / 2 ? candidate : null;
}

```

6. Conclusion

The Boyer–Moore Majority Vote algorithm demonstrates **optimal asymptotic efficiency** with $\mathbf{O(n)}$ time and $\mathbf{O(1)}$ space complexity across all cases.

The primary correctness enhancement is adjusting the iteration to begin at $i = 1$. Further improvements—such as disabling tracker increments during timed benchmarks and restructuring conditions for branch efficiency—refine performance without affecting asymptotic growth.

Empirical validation through benchmarking should confirm **linear scaling** and showcase the efficiency gains after code refinement.

Overall, the implementation achieves theoretical optimality while maintaining clear, maintainable, and testable code.